

External Sorting



DreamStudio prompt: old man sorting numbers

Updates

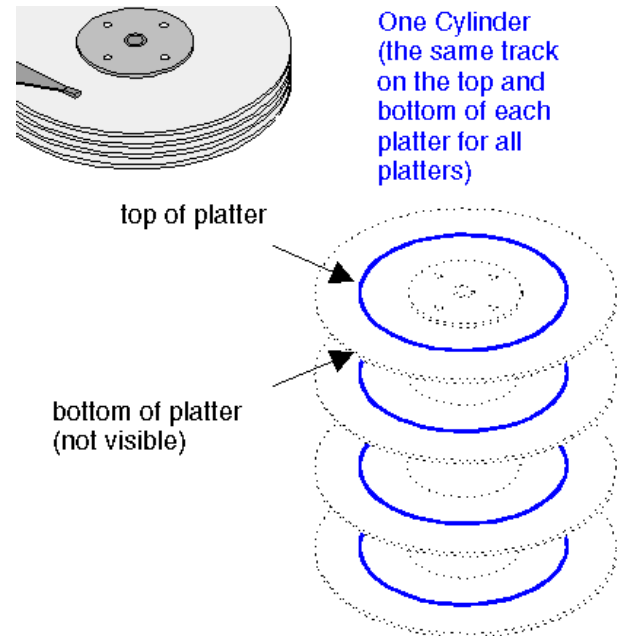
❖ Today:

- External Sorting
- Quiz 2
 - Must be done in class
 - On Canvas

❖ Project 1 Deadline: Apr 3

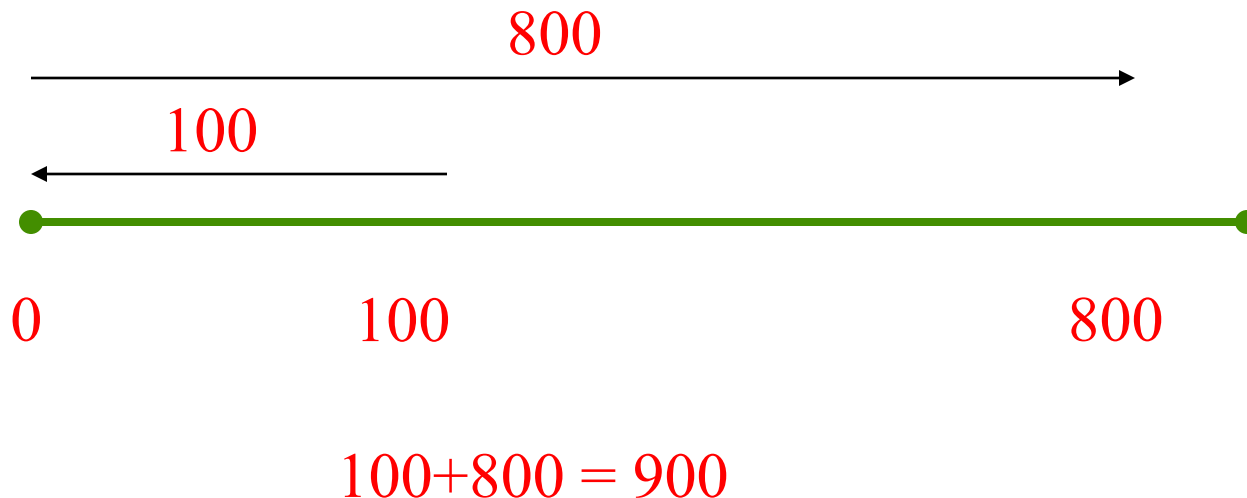
Quiz 1 Discussion

- ❖ If one disk track can hold up to 200 blocks, and the disk has 3 platters, then **one cylinder** can hold up to how many disk blocks?
- ❖ 2 surfaces, 3 platters, 200 blocks each
- ❖ $2 * 200 * 3 = 1200$



Quiz 1 Discussion

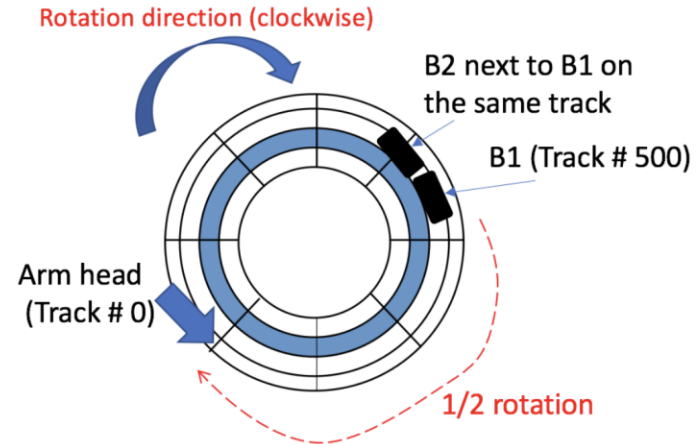
- ❖ Requests to be served: [500, 70, 30, 5, 800, 10],
- ❖ Arm head is at track #100



Quiz 1 Discussion

Disk Configuration

- Disk rotates clockwise
- Disk rotates at 6000 RPM
- Seek time: 1 ms (warm-up), + 1ms for every 500 cylinders traveled.
- Reading one disk block requires 0.5 ms



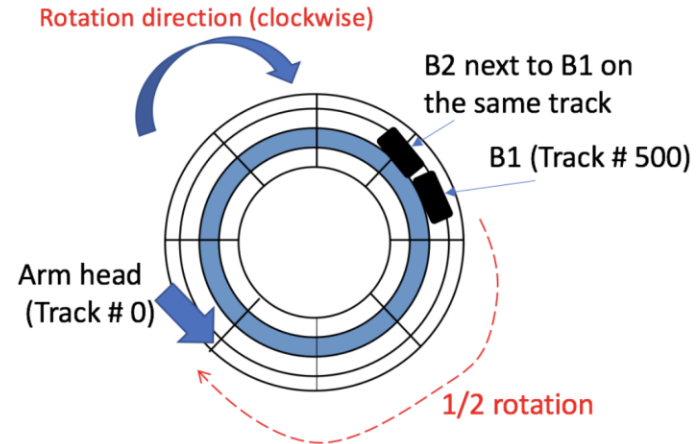
❖ block B1 followed by block B2

$$\begin{aligned} &= \text{Warm-up} + \text{Time to move B1} + \text{Time for half rotation} + \\ &\quad \text{Time to read block B1} + \text{Time to read block B2} \\ &= 1 \text{ (warm-up)} + 1 \text{ ms (seek time)} + 5 \text{ ms (half} \\ &\quad \text{rotation)} + 1 \text{ ms (transfer time B1+B2)} \\ &= 8 \text{ ms} \end{aligned}$$

Quiz 1 Discussion

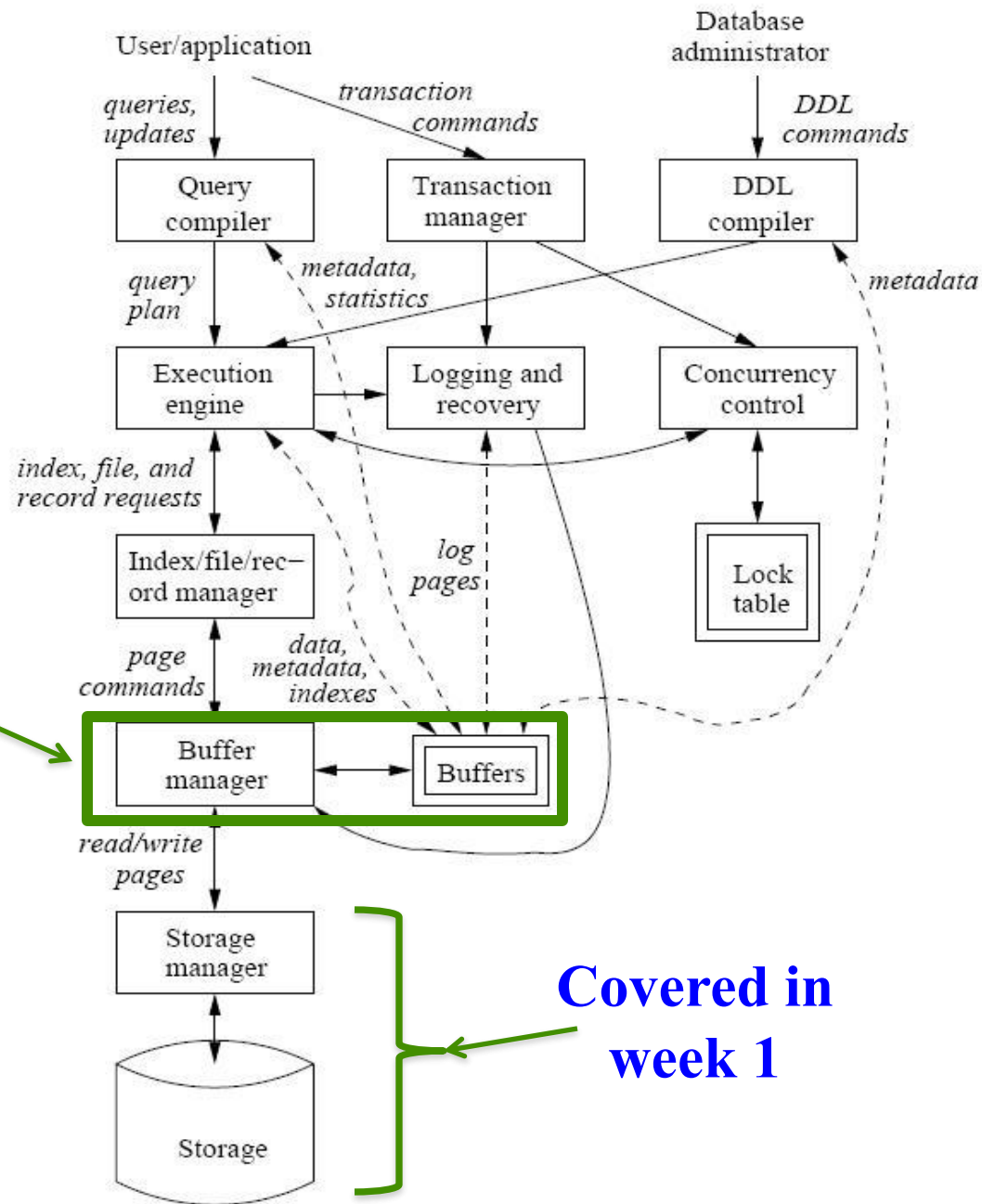
Disk Configuration

- Disk rotates clockwise
- Disk rotates at 6000 RPM
- Seek time: 1 ms (warm-up), + 1ms for every 500 cylinders traveled.
- Reading one disk block requires 0.5 ms



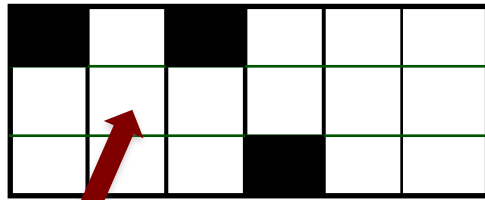
❖ block B2 followed by block B1

$$\begin{aligned} &= 1 \text{ (warm-up)} + 1 \text{ ms (seek time)} + 5 \text{ ms (half rotation)} + 0.5 \text{ ms (transfer time B1)} + \mathbf{10 \text{ ms (full rotation to get to B2)}} + 0.5 \text{ ms (transfer time B2)} \\ &= \mathbf{18 \text{ ms}} \end{aligned}$$



Database management system components

Buffer Pool



frame

Need record
rid= (block i , slot j)

Extract the block Id from rid $\rightarrow i$

No

Is block i in
buffer pool?

Is there an
Empty frame?

No

Can I free any of
Existing frames?

No

- Request cannot be served now
- Queue the request

Yes
(Frame k)

For this frame,
is "dirty" flag= True?

Yes

Write the block in
Frame k back to disk

No

Get block i from disk

- Put block i in frame k
- Fill in the frame metadata

- Extract the slotId from rid $\rightarrow j$
- Read the record and return it to the caller

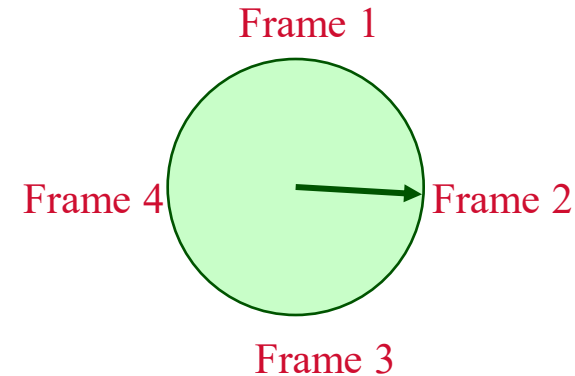
LRU Replacement Policy

❖ *Least Recently Used (LRU)*

- for each frame in buffer pool, keep track of time when last *accessed* → (*LTtimestamp*)
- replace the frame which has the oldest (earliest) time
- very common policy: intuitive and simple
 - Works well for repeated accesses to popular pages

“Clock” Replacement Policy

- ❖ An approximation of LRU
- ❖ Each frame has
 - Pin count → If larger than 0, do not touch it
 - Second chance bit (Ref) → 0 or 1
- ❖ A pointer rotates to find a candidate frame to free
- ❖ **READ/WRITE page already in buffer → Set (Ref = 1)**

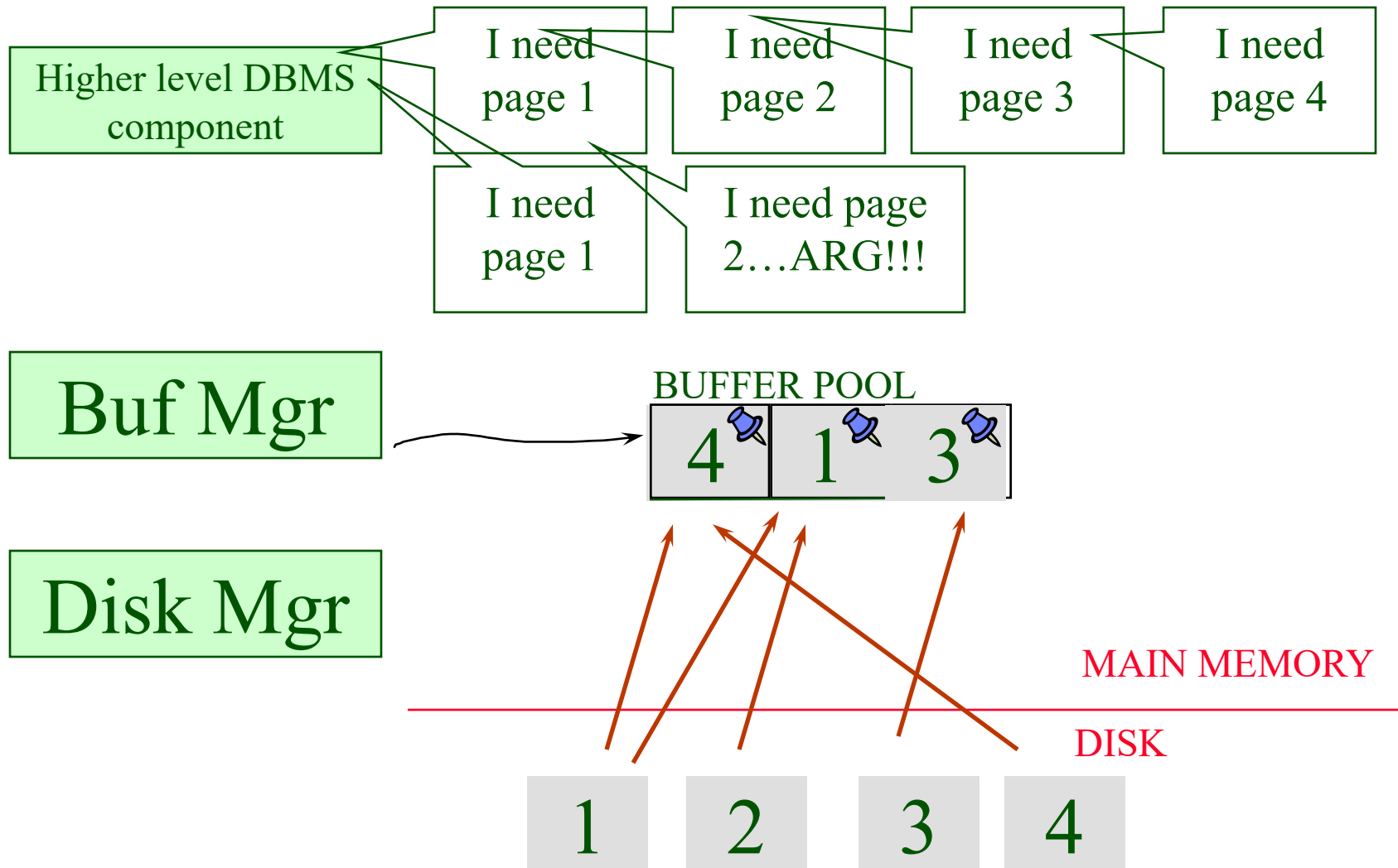


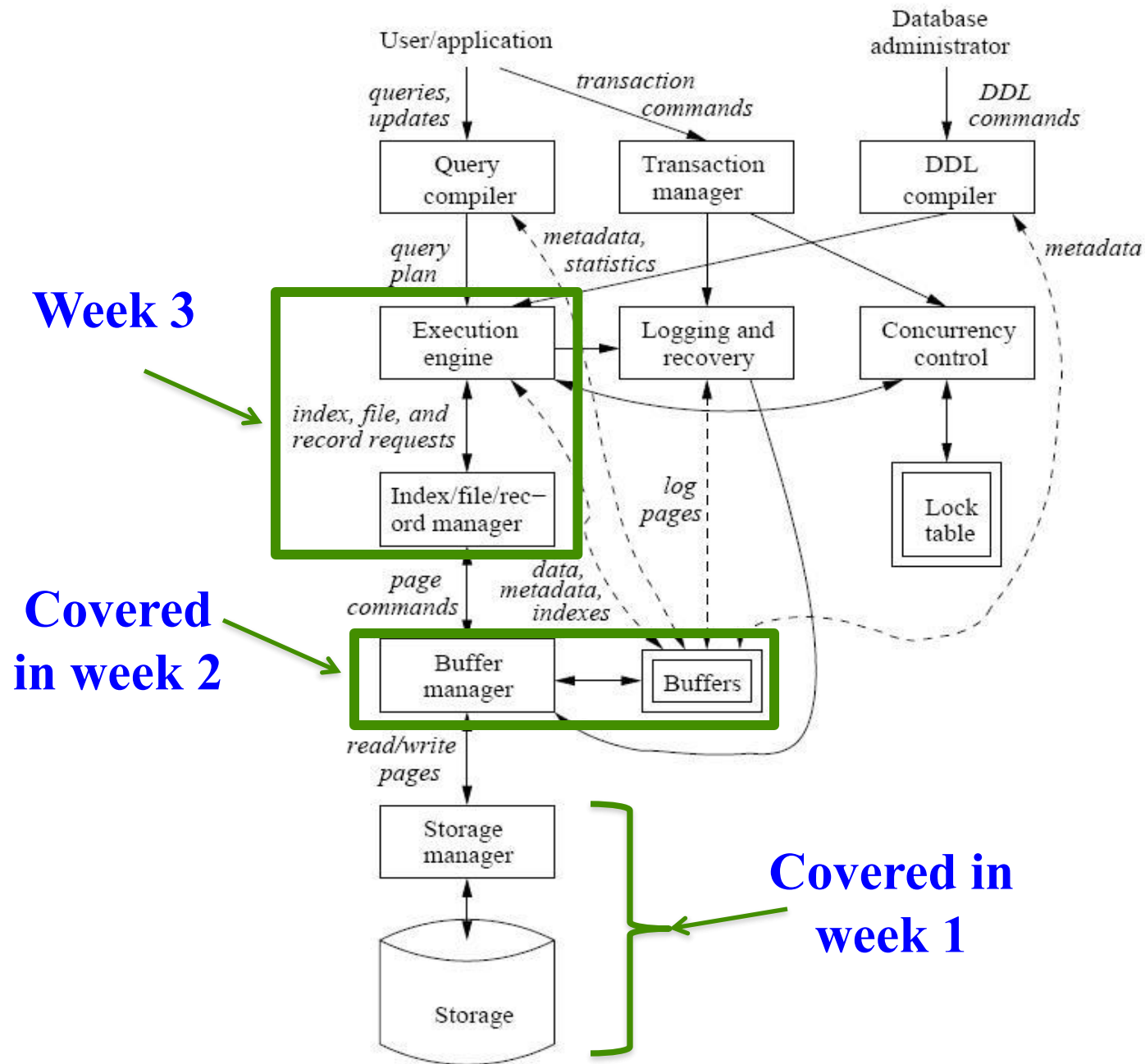
IF pin-count > 0 Then → Skip

IF (pin-count = 0) & (Ref = 1) → Set (Ref = 0) and skip (second chance)

IF (pin-count = 0) & (Ref = 0) → free and re-use

Sequential flooding example with LRU





Learning Objectives

- ❖ Understand the role of sorting in DB
- ❖ Be able to implement 2-way and M-way merge sort
- ❖ Explain how MergeSort can leverage buffers to sort files much larger than memory
- ❖ Use eqs. to compute #passes in external merge sort given N pages and M buffers

Sorting in main-memory

- ❖ Merge Sort

- ❖ Heap Sort

- ❖ Quick Sort

In DBMSs, the data can be much bigger than the main memory

Why Sort?

- ❖ Data **requested** in sorted order (Explicit)
 - e.g., find students in increasing *gpa* order
 - ORDER BY clause in SQL
- ❖ Many DB operations need sorting (Implicit)
 - Grouping, duplicate elimination, joins
 - Creating indexes
- ❖ **Problem: sort 10GB of data with 50MB of RAM.**

Using secondary storage effectively

❖ General Wisdom:

- I/O costs dominate
- Design algorithms to reduce I/O
- CPU overhead becomes less important

We Care About I/O Cost

Efficient Data Access Matters



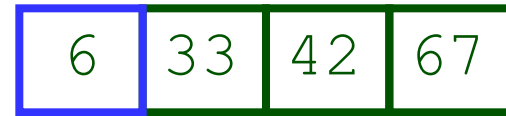
Part I

How to Merge Two Sorted Lists

Let's forget the “big data” for now and think about arrays

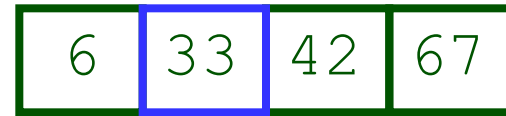
How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



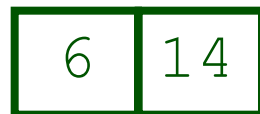
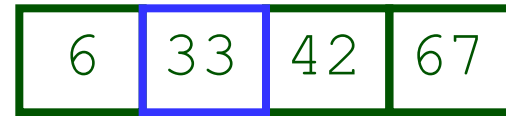
How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



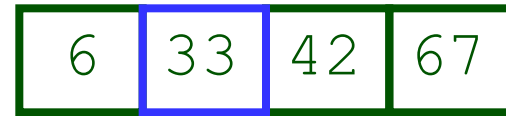
How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



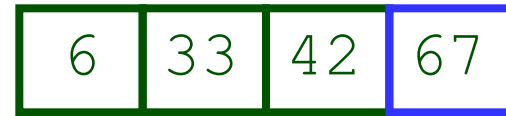
How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



Merge

How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer

Do I need to “see” anything other than what is in blue?

14	23	45	98
----	----	----	----

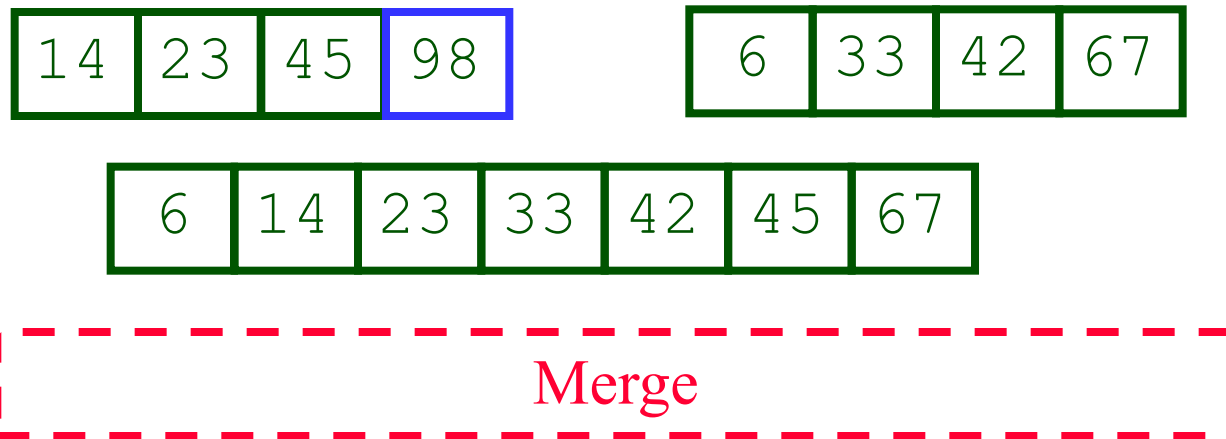
6	33	42	67
---	----	----	----

6	14	23	33	42	45
---	----	----	----	----	----

Merge

How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer



How to Merge Two Sorted Lists

- ❖ $O(n+m)$, n & m are the lengths of the two lists
- ❖ Keep a pointer at the beginning of each list
- ❖ Move the smaller value to the output and advance the pointer
- ❖ ****Linear scan of the two lists****

What is assumed about the inputs?

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45	67	98
---	----	----	----	----	----	----	----

Merge

Part II: External Merge Sort

How to do it externally (using disk)

How to do it externally (using disk)

```
SELECT *  
FROM Deposits  
ORDER BY date;
```

1,000s of disk blocks



Two main questions

- How many memory buffers are needed ?
- What is the I/O cost ?

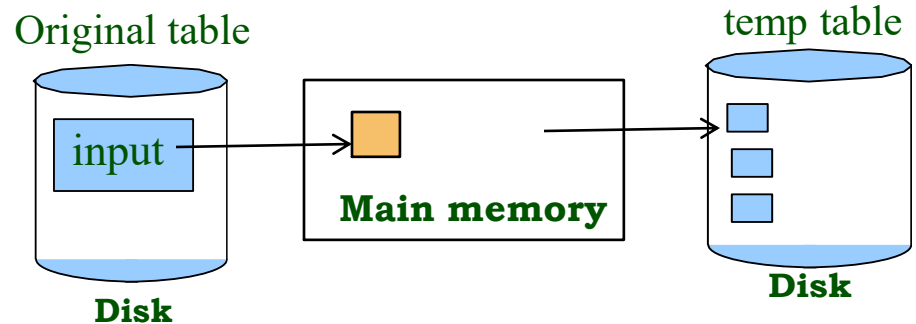
Let's think about this problem...

- ❖ The file consists of MANY blocks.
- ❖ What if the file is not sorted?
- ❖ How much memory is needed to sort 1 block?
- ❖ How much memory is needed to merge 2 sorted blocks?

2-Way External Sort (Naïve way)

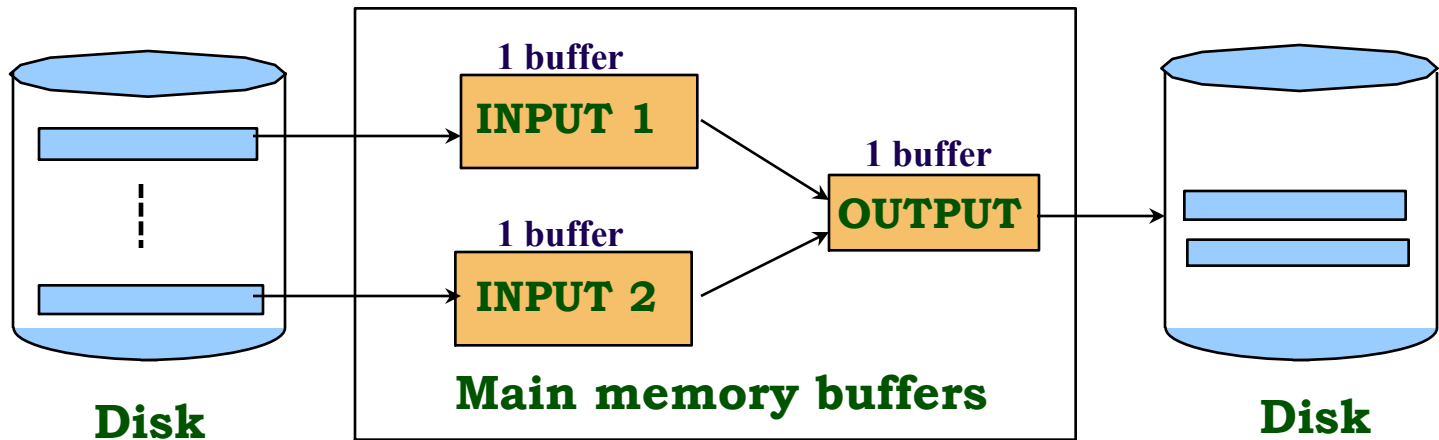
❖ Phase 1: PREPARE.

- Read a page, sort it, write it.
- only one buffer page is used



❖ Phase 2, 3, ..., etc.: MERGE two sub-lists in each phase

- Three buffer pages used.



Two-Way External Merge Sort

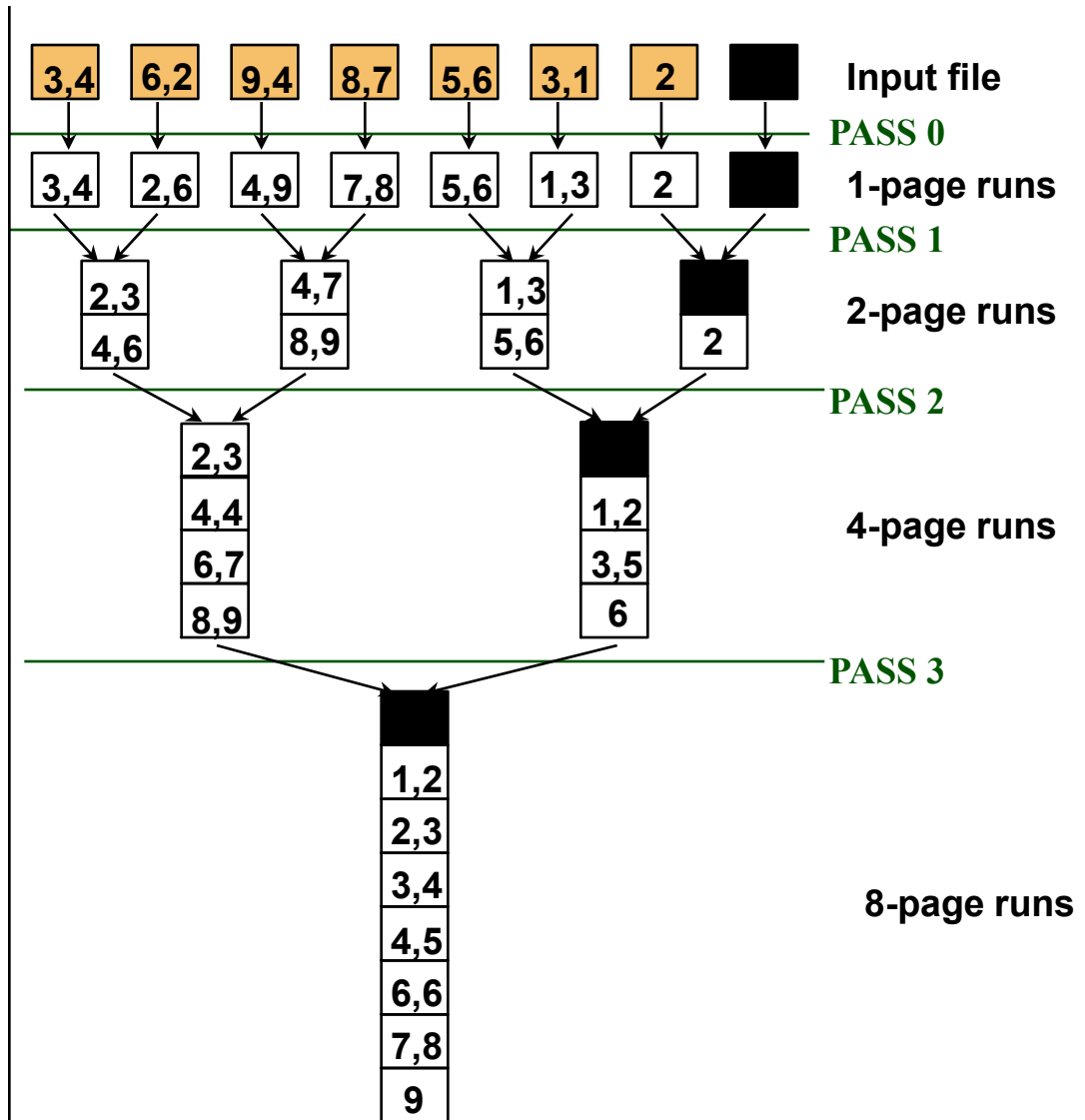
- ❖ Idea: *Divide and conquer*: sort subfiles and merge into larger sorts

Pass 0 → Only 1 memory block is needed

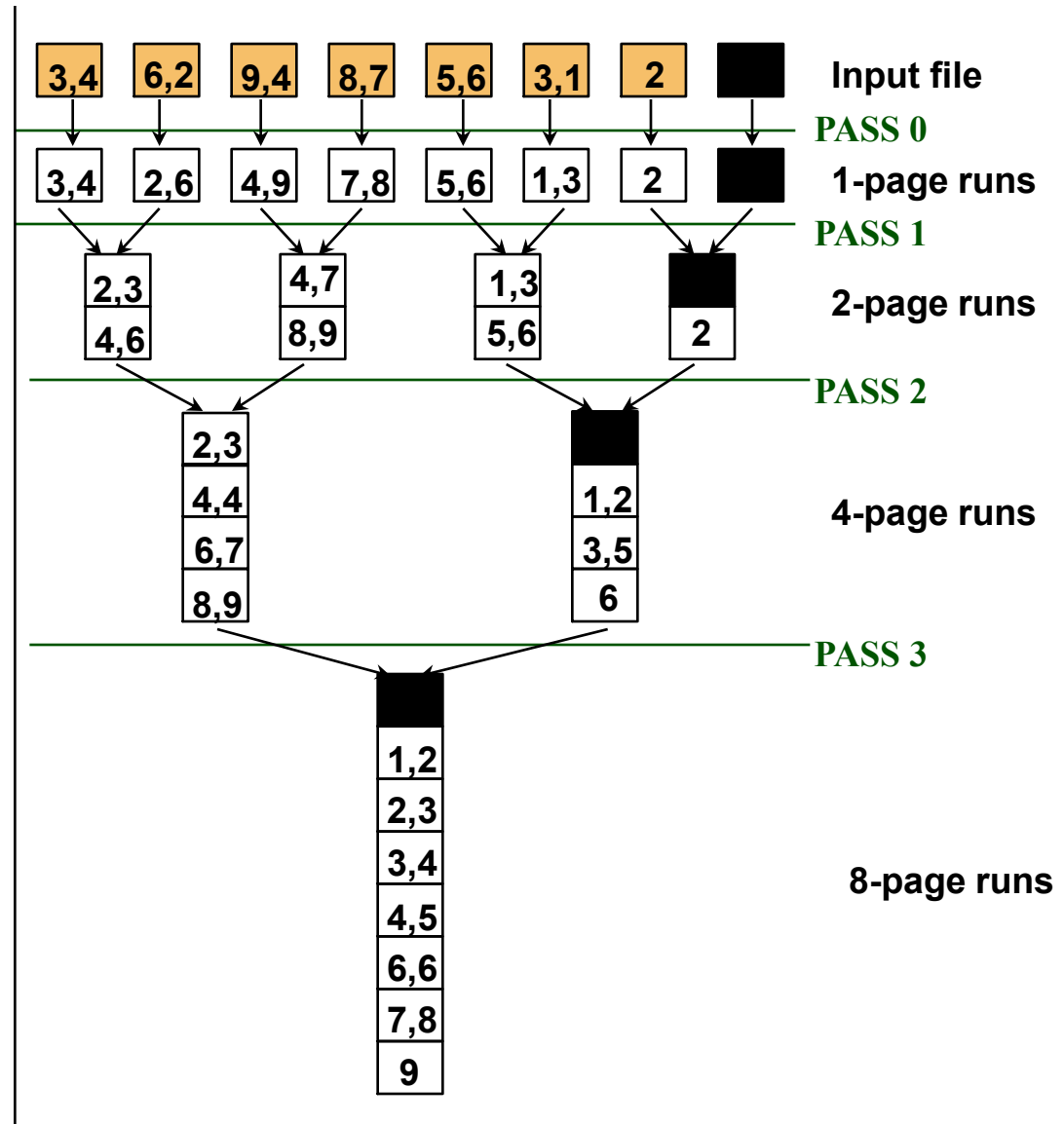
Pass $i > 0$ → Only 3 memory blocks are needed

3,4 2,6

Buffer: 3,4 2,6



Two-Way External Merge Sort



❖ **Costs for one pass:**
all pages

❖ **# of passes :**
height of tree

❖ **Total cost :**
product of above

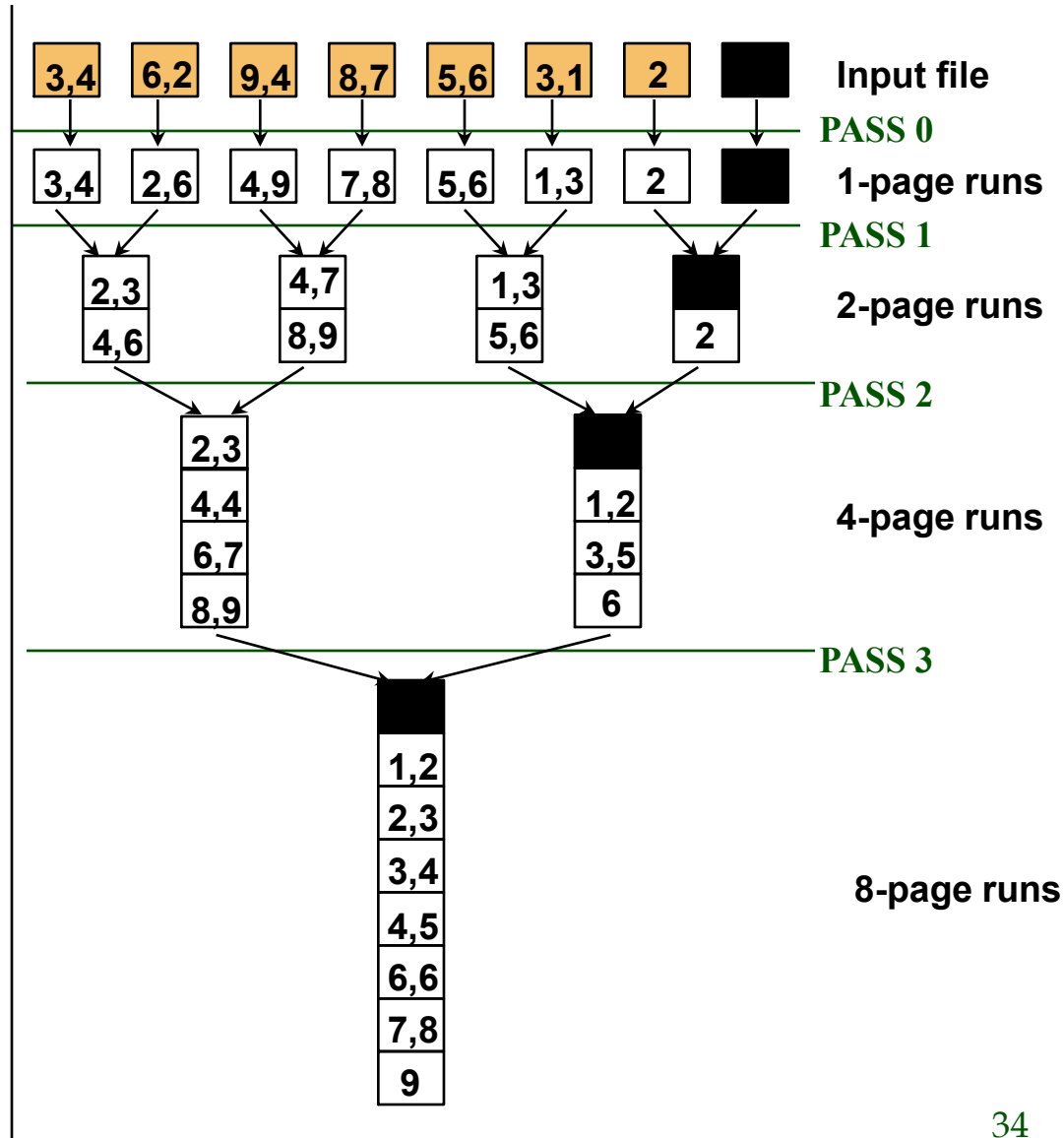
NB: We ignored the CPU cost to sort a block in memory or merge two blocks

Two-Way External Merge Sort

- ❖ Each pass we read + write each page in file.
- ❖ N pages in file $\rightarrow 2N$
- ❖ Number of passes

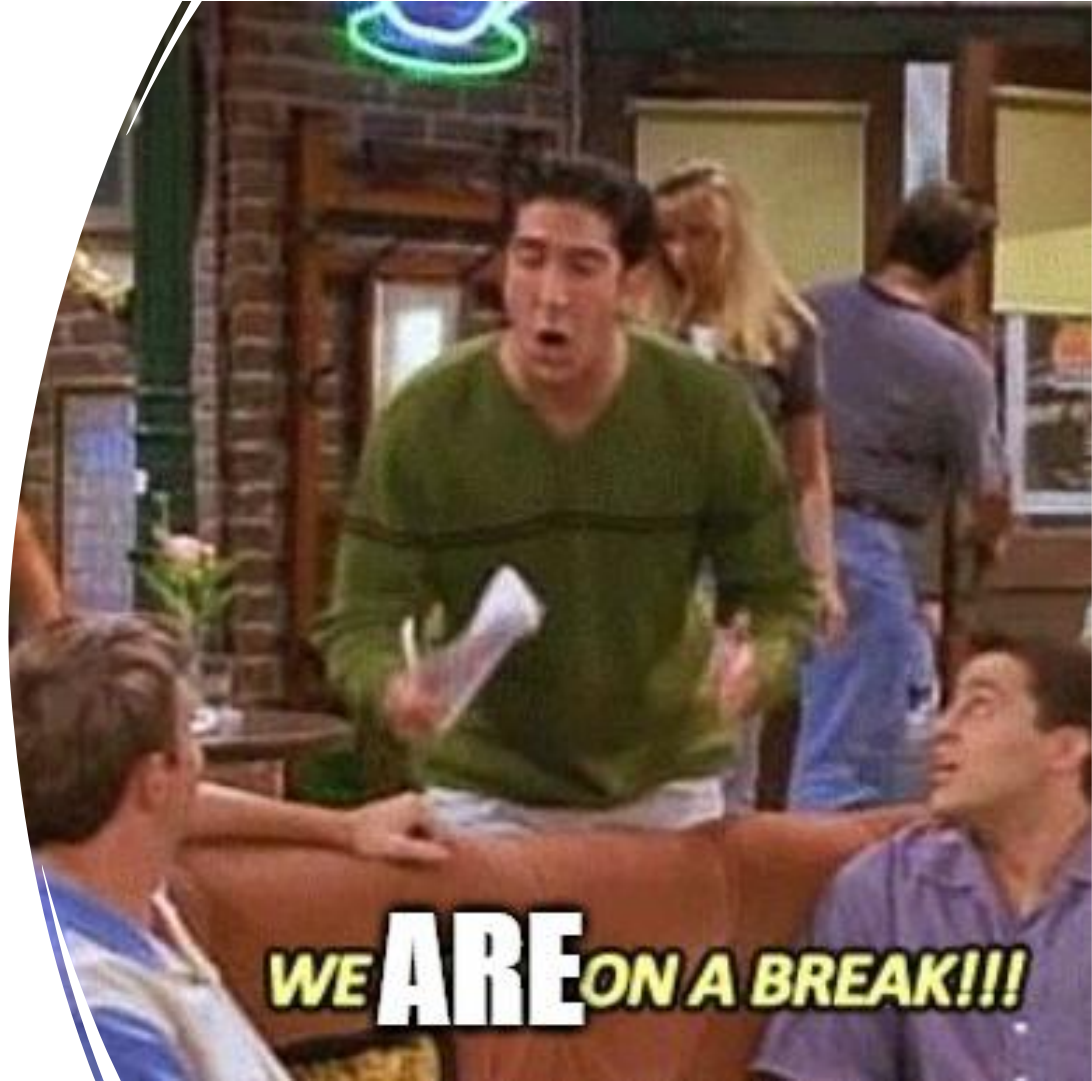
$$= \lceil \log_2 N \rceil + 1$$
- ❖ So total I/O cost is:

$$2N(\lceil \log_2 N \rceil + 1)$$



Break

- ❖ Now we will take a few minutes break
- ❖ Make sure we have everything you need for the **Quiz**

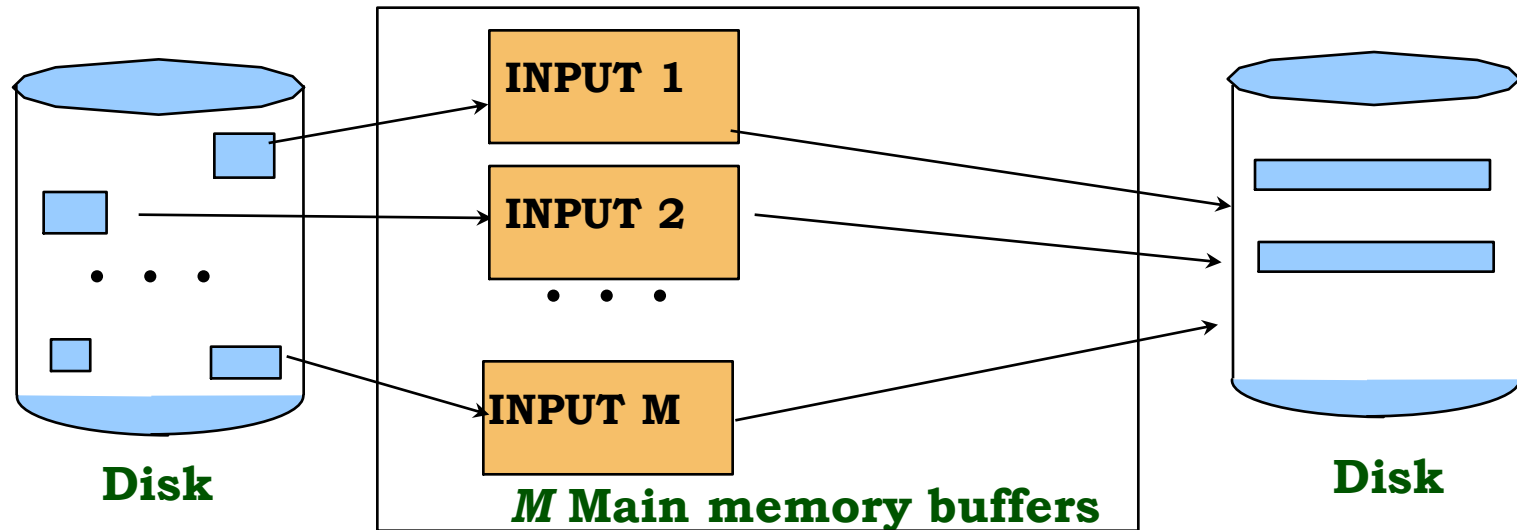


M-way External Merge Sort

- ❖ What if we had more buffer pages (*M buffers*)?
- ❖ How to utilize them wisely to reduce I/O?

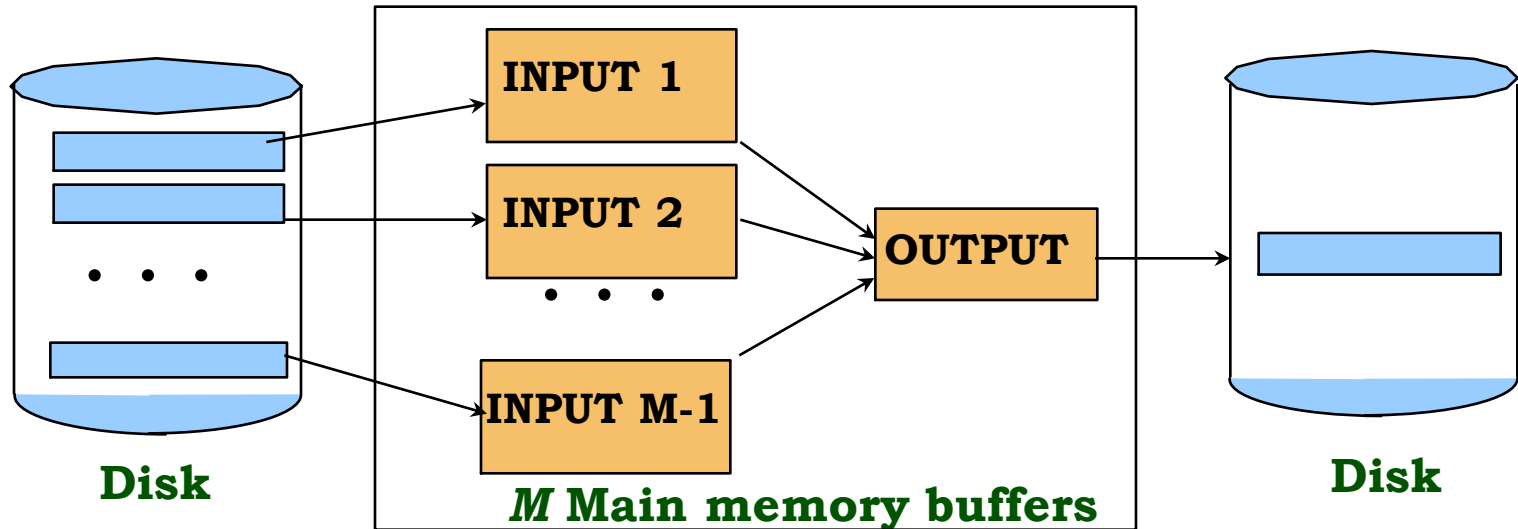
→ Two main ideas !

Phase 1 : Prepare (idea 1)



- Construct as large as possible starter lists.
- Sort M blocks at once (in-place algorithm)
 - **Output: ONE SORTED LIST composed by M blocks**
- ➔ Will reduce the number of needed passes

Phases 2, 3, ... : Merge (idea 2)



(list = run)

- ❖ Merge as many sorted sublists into one long sorted list.
- ❖ Keep 1 buffer for the output
- ❖ Use M-1 buffers to read from M-1 lists



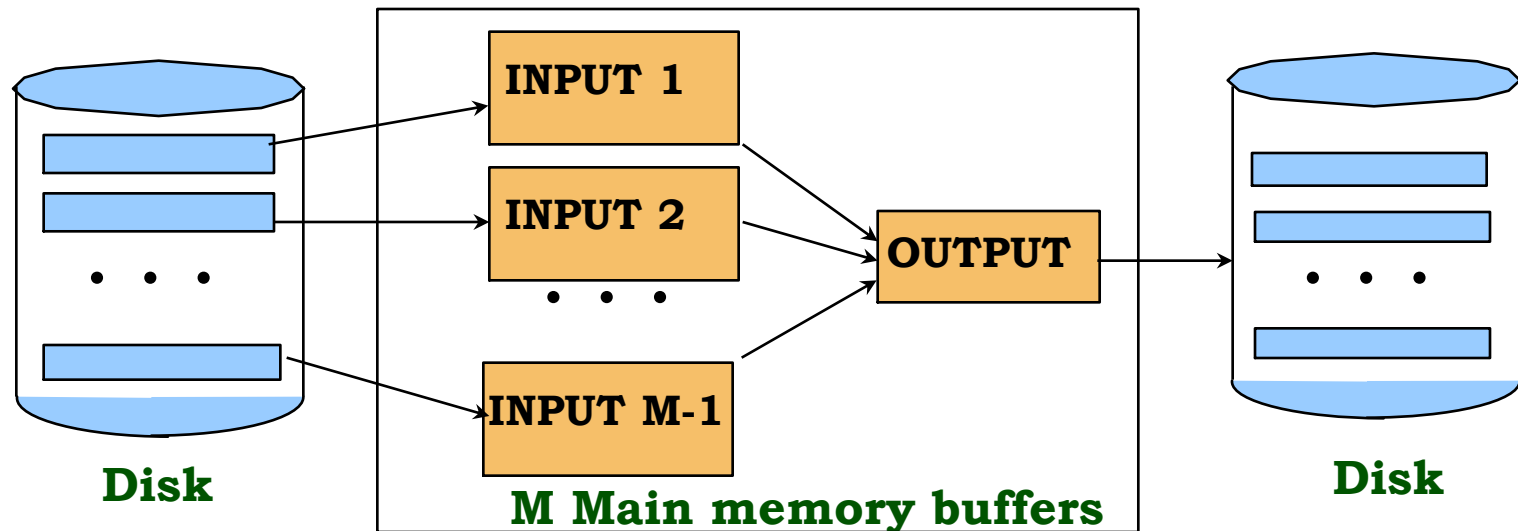
“M”-way might be misleading
Different than 2-way case

General External Merge Sort

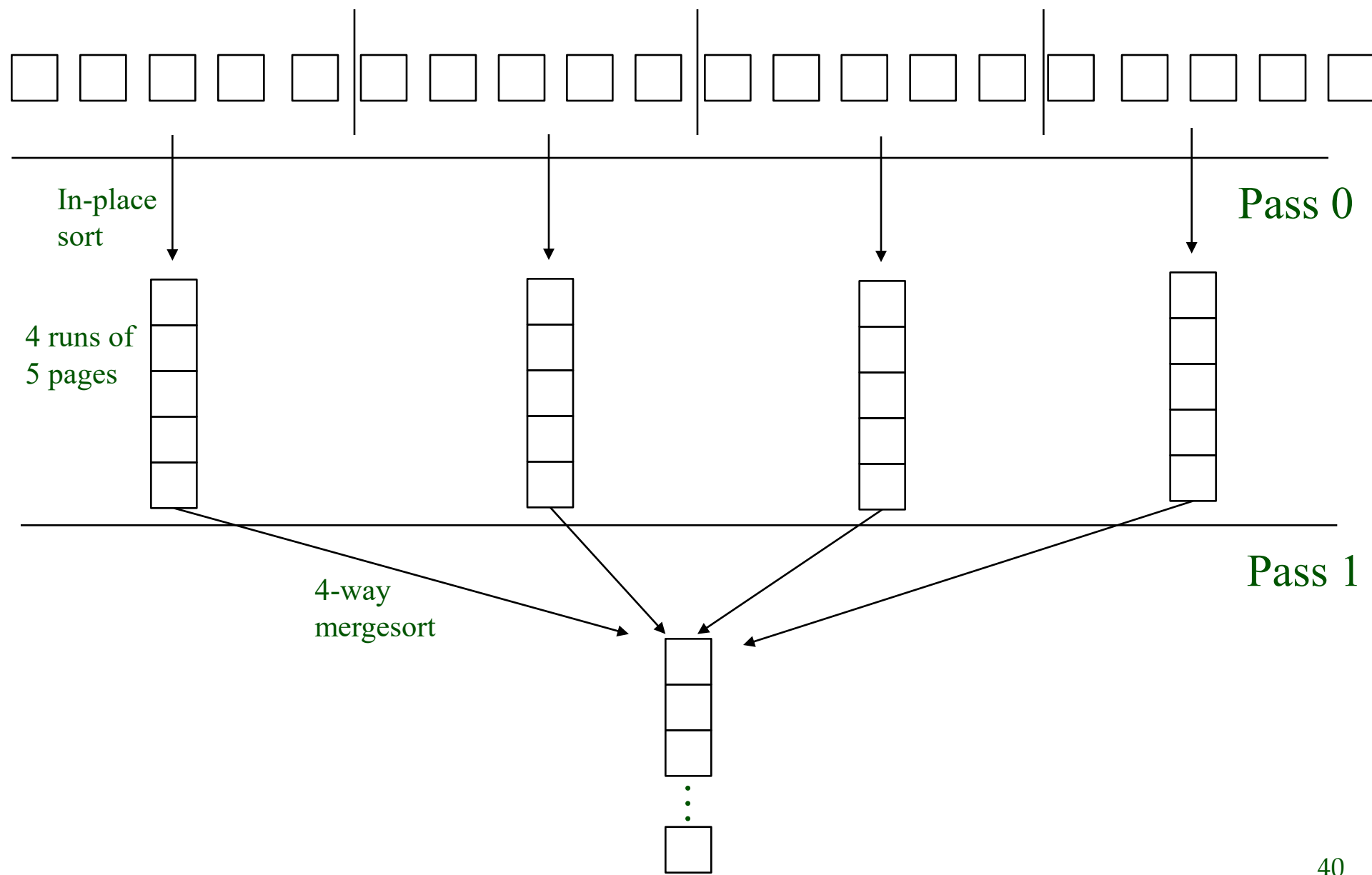
□ *How can we utilize more than 3 buffer pages?*

❖ To sort a file with N pages using M buffer pages:

- **Pass 0: use M buffer pages.**
Produce N/M sorted runs of M pages each.
- **Pass 1, 2, ..., etc.: merge $M-1$ runs.**



Example with $N=20$, $M=5$



Cost of External Merge Sort

- ❖ Number of passes: $1 + \lceil \log_{M-1} \lceil N / M \rceil \rceil$
- ❖ Total I/O Cost = $2N * (\text{\# of passes})$

Example

- ❖ Buffer : with $M=5$ buffer pages
- ❖ File to sort : $N=108$ pages
- Pass 0:
 - Size of each run?
 - Number of runs?
- Pass 1:
 - Size of each run?
 - Number of runs?
- Pass 2: ???

Example

❖ Buffer : with $M=5$ buffer pages

❖ File to sort : $N=108$ pages

- Pass 0: $\lceil 108 / 5 \rceil = 22$ sorted runs of 5 pages each (last run is only 3 pages)
- Pass 1: $\lceil 22 / 4 \rceil = 6$ sorted runs of 20 pages each (last run is only 8 pages)
- Pass 2: 2 sorted runs, 80 pages and 28 pages
- Pass 3: Sorted file of 108 pages

• Total I/O costs: ?

Example

❖ For 5-way sorting

- Total I/O costs: $2 \times 10^8 \times (4 \text{ passes})$

❖ What if we did 2-way sorting (w/ inefficient Phase 1)?

- Total I/O costs: $2 \times 10^8 \times (8 \text{ passes})$

Number of Passes of External Sort

- gain of utilizing all available buffers
- importance of a high fan-in during merging

256 x 4KB
= 1MB

#Buffers available in main-memory

#Pages
in File

N	M=1	M=5	M=9	M=17	M=129	M=257
100	7	4	3	2	1	1
1,000	10	5	4	3	2	2
10,000	13	7	5	4	2	2
100,000	17	9	6	5	3	3
1,000,000	20	10	7	5	3	3
10,000,000	23	12	8	6	4	3
100,000,000	26	14	9	7	4	4
1,000,000,000	30	15	10	8	5	4

Summary: External Sorting

- ❖ Use as much memory buffers you can
- ❖ Cost metric ?
 - I/O only (till now)
 - CPU is nontrivial, worth reducing
 - Use a quick main-memory algorithm
- ❖ For the in-memory sorting part
 - Use *Quicksort* or *heapsort*

Quiz 2

- **Reminders:**
 - **Closed book**
 - **Individual work**
 - **You have 20 minutes**

